# PRD — FocaQuest

## Plataforma de Aprendizagem Adaptativa para Concursos Públicos

**Versão:** 2.0
**Data:** 21/10/2026
**Status:** Pronto para detalhamento técnico e prototipação
**Primeiro concurso:** TJPB 2026
**Especialidade inicial:** Analista TI — Desenvolvimento de Sistemas
**Banca:** FGV
**Produto inicial:** Aplicação web responsiva (PWA)

---

# 1. Visão do produto

O FocaQuest é uma plataforma de aprendizagem que organiza a preparação para concursos públicos em pequenas experiências de estudo interativas, utilizando metodologias de aprendizagem ativa, revisões espaçadas, personalização e gamificação.

A experiência deverá favorecer a facilidade de iniciar atividades, a manutenção do engajamento e a retomada dos estudos após interrupções.

O produto terá recursos opcionais de acessibilidade cognitiva, atendendo estudantes com diferentes estilos e necessidades de atenção, sem exigir a declaração de diagnóstico.

### 1.1 Diferencial competitivo

O FocaQuest não será apenas um banco de questões ou um sistema de cronogramas.

Sua proposta será integrar quatro componentes:

1. **Tutor de aprendizagem:** ensina um conceito com explicações, exemplos e exercícios.
2. **Motor adaptativo:** identifica tópicos que precisam de reforço e escolhe atividades apropriadas.
3. **Assistente de foco:** reduz o esforço de começar, continuar e retomar uma sessão.
4. **Preparação por edital:** organiza o conteúdo conforme concurso, cargo, banca e formato da prova.

### 1.2 Proposta central

Ao abrir o aplicativo, o estudante deverá encontrar uma atividade recomendada e conseguir iniciar o estudo imediatamente.

O aplicativo deverá responder automaticamente às perguntas:

- O que estudar agora?
- O que já aprendi?
- O que ainda preciso reforçar?
- Quais conteúdos precisam ser revisados?
- Quanto do edital já percorri?
- Como está meu desempenho nas questões?

---

# 2. Estrutura curricular

## 2.1 Modelo curricular

Exam → ExamRole → SyllabusVersion → Subject → Topic → LearningObjective → Lesson → Activity

Cada item curricular deverá ter:

- Identificador estável.
- Referência ao concurso e à versão do edital.
- Nome e descrição.
- Hierarquia de assuntos.
- Objetivos de aprendizagem.
- Status de cobertura por conteúdos e questões.
- Referência à fonte oficial.
- Data da última revisão editorial.

## 2.2 Concurso inicial

### Conhecimentos Gerais

- Língua Portuguesa — 10 questões.
- Raciocínio Lógico e Matemático — 10 questões.

### Conhecimentos Específicos — 40 questões

- Fundamentos de TI comuns às especialidades.
- Engenharia de Software e Práticas Ágeis.
- Design de Sistemas.
- Arquitetura de Software e Padrões.
- Algoritmos e Programação.
- Desenvolvimento Mobile.
- Banco de Dados e Persistência.
- DevOps, Infraestrutura como Código e Nuvem.
- Inteligência Artificial.

- Segurança da Informação e Qualidade de Software.
- Governança e Regulação do Poder Judiciário.
- Design de Interface e Experiência do Usuário.

Os conteúdos devem ser cadastrados individualmente, preservando todos os subitens exigidos no edital. Os agrupamentos apresentados são pedagógicos, não necessariamente disciplinas independentes na prova.

### 2.3 Priorização curricular

O sistema deverá permitir configurar diferentes estratégias:

- Seguir a ordem do edital.
- Priorizar conteúdos ainda não estudados.
- Priorizar conteúdos com baixo desempenho.
- Estudar por disciplina.
- Revisar conteúdos anteriores.
- Treinar assuntos associados a determinada banca.

No MVP, a recomendação poderá utilizar regras explícitas. Algoritmos mais sofisticados serão uma evolução posterior.

---

## 3. Experiência principal

### 3.1 Tela inicial — Minha jornada

A página inicial deverá apresentar:

- Missão recomendada.
- Concurso e cargo selecionados.
- Progresso curricular.
- Revisões pendentes.
- Última atividade interrompida.
- Acesso rápido ao mapa de aprendizagem.

A ação principal será o botão **Começar missão**.

### 3.2 Tipos de missão

**Micro:** aproximadamente 3–5 minutos.

**Padrão:** aproximadamente 8–12 minutos.

**Aprofundada:** aproximadamente 15–25 minutos.

A duração será indicativa e nunca uma obrigação.

### 3.3 Estrutura de uma microlição

Cada microlição deverá conter:

1. Apresentação do objetivo.
2. Explicação curta do conceito.
3. Exemplo aplicado.
4. Atividade de recuperação.
5. Feedback comentado.
6. Registro do resultado.
7. Recomendação do próximo passo.

### 3.4 Questões contextualizadas

As atividades poderão apresentar situações de desenvolvimento de software, administração pública, segurança, sistemas e outros cenários relacionados ao concurso.

Exemplo:

**Tema:** Banco de Dados — Normalização.

**Situação:** uma aplicação armazena dados de clientes e pedidos na mesma tabela.

**Missão:** identificar problemas de redundância e escolher a alternativa que representa uma modelagem adequada.

O sistema deverá justificar a alternativa correta e explicar os problemas das incorretas.

---

## 4. Aprendizagem adaptativa

### 4.1 Estados de aprendizagem

Cada tópico poderá assumir os seguintes estados:

- Não iniciado.
- Em aprendizagem.
- Precisa de reforço.
- Em revisão.
- Domínio estimado.

O estado deverá ser calculado a partir de evidências de aprendizagem, não simplesmente de lições visualizadas.

### 4.2 Motor inicial de recomendação

O motor de recomendação deverá considerar:

- Assuntos disponíveis no currículo.
- Questões respondidas.
- Acertos e erros recentes.
- Revisões vencidas.
- Conteúdos ainda não estudados.
- Disponibilidade de tempo informada pelo usuário.

Regras iniciais:

1. Priorizar revisões relevantes já programadas.
2. Alternar, quando apropriado, revisão e novos conteúdos.
3. Reforçar assuntos com respostas incorretas.
4. Evitar repetições excessivas de uma mesma questão.
5. Oferecer sempre uma alternativa de missão curta.

### 4.3 Revisão espaçada

O MVP utilizará um escalonador determinístico e testável de revisões, com intervalos calculados segundo o resultado da recuperação.

O escalonador deverá permitir modificar sua política sem perder o histórico de tentativas.

---

## 5. Recursos de atenção e acessibilidade

### 5.1 Modo foco

Quando ativado:

- Oculta informações periféricas.
- Reduz elementos de navegação.
- Mostra uma tarefa por vez.
- Desativa animações não essenciais.
- Permite pausar ou encerrar a atividade.
- Mantém o progresso salvo.

### 5.2 Retomada inteligente

Quando um estudante retorna após uma interrupção, o sistema deverá apresentar:

**Você parou neste desafio. Deseja continuar?**

Não será necessário reiniciar a lição ou refazer questões já respondidas.

### 5.3 Metas sem punição

- Ausências não geram perda de progresso.
- Sequências de dias são opcionais.

- Recompensas devem enfatizar aprendizagem e retomadas.
- Temporizadores de estudo são opcionais.
- Notificações poderão ser completamente desativadas.

---

# 6. Gamificação

### MVP

- XP por conclusão de atividades.
- Indicadores de progresso curricular.
- Feedback visual após uma missão.
- Marcos de domínio por assunto.

### Evolução

- Mapas de progressão.
- Conquistas e coleções.
- Avatar personalizável.
- Desafios especiais.
- Eventos temáticos.

XP e domínio serão métricas separadas.

---

# 7. Questões e simulados

## 7.1 Questões objetivas

Cada questão deverá possuir:

- Enunciado.
- Cinco alternativas no formato FGV, quando aplicável.
- Alternativa correta.
- Explicação comentada.
- Assunto associado.
- Dificuldade editorial.
- Fonte e autoria.
- Histórico de revisão.
- Estado de publicação.

## 7.2 Simulado

O sistema deverá permitir selecionar uma prova-modelo do TJPB, contendo:

- 10 questões de Português.
- 10 questões de Raciocínio Lógico e Matemático.

- 40 questões de conhecimentos específicos.

O resultado deverá apresentar pontuação geral e desempenho por área.

### 7.3 Discursiva — Evolução prioritária

Será previsto um módulo de treinamento de respostas discursivas com:

- Tema relacionado ao edital.
- Editor de texto.
- Contagem de linhas simulada.
- Critérios de avaliação configuráveis.
- Registro de tentativas.
- Autoavaliação orientada.

A correção automatizada por IA, quando implementada, será apenas formativa e não representará a correção oficial da FGV.

---

## 8. Administração de conteúdos

O produto deverá possuir um mecanismo para inserir conteúdos revisados.

No MVP, o conteúdo poderá ser importado por arquivos estruturados versionados no Git.

Requisitos:

- Importação idempotente.
- Identificação de versões.
- Rastreabilidade de fonte.
- Validação de alternativas e gabaritos.
- Integridade das referências curriculares.
- Rejeição de questões incompletas.

Um painel administrativo completo não é necessário para a primeira versão.

---

## 9. Requisitos de qualidade

- Aplicação responsiva e compatível com dispositivos móveis.
- Acessibilidade orientada pela WCAG 2.2 AA.
- Interface utilizável com teclado.
- Persistência de progresso confiável.
- Controle de acesso aos dados dos usuários.
- Validação no backend.
- Tratamento de erros e estados de carregamento.
- Código versionado e documentado.

- Testes de regras críticas.
- Separação entre conteúdo, regras pedagógicas e apresentação.
- Observabilidade básica.
- Respeito à LGPD e aos direitos autorais.

---

## 10. Critério de sucesso do MVP

O MVP será considerado funcional quando um estudante conseguir:

1. Selecionar o TJPB e Desenvolvimento de Sistemas.
2. Iniciar uma missão.
3. Aprender um conceito.
4. Resolver questões relacionadas.
5. Receber feedback comentado.
6. Interromper e retomar a sessão.
7. Consultar seu progresso.
8. Executar uma revisão recomendada.
9. Utilizar a experiência em computador e celular.

A conclusão dessas etapas deverá ser validada por testes automatizados e por pelo menos um teste completo com usuário.

---

## 11. Limites do MVP

A primeira versão não terá:

- Tutoria por IA.
- Importação automática de PDF.
- Aplicativo nativo.
- Comunidade e ranking global.
- Marketplace.
- Monetização.
- Simulados discursivos automatizados.
- Geração ilimitada de questões.

O objetivo é demonstrar um ciclo completo de aprendizagem e validar sua utilidade antes da expansão.

# 5. Arquitetura técnica recomendada

A stack principal será a seguinte:

| Camada | Tecnologia |
|---|---|
| Frontend | React, Next.js, TypeScript |
| Estilização | Tailwind CSS |
| Backend | Java 21+, Spring Boot |
| Segurança | Spring Security |
| Persistência | PostgreSQL, Spring Data JPA |
| Migrations | Flyway |
| Testes | JUnit, Mockito, Testcontainers, Vitest |
| Documentação | API OpenAPI / Swagger |
| Infraestrutura | Docker Compose |
| CI/CD | GitHub Actions |

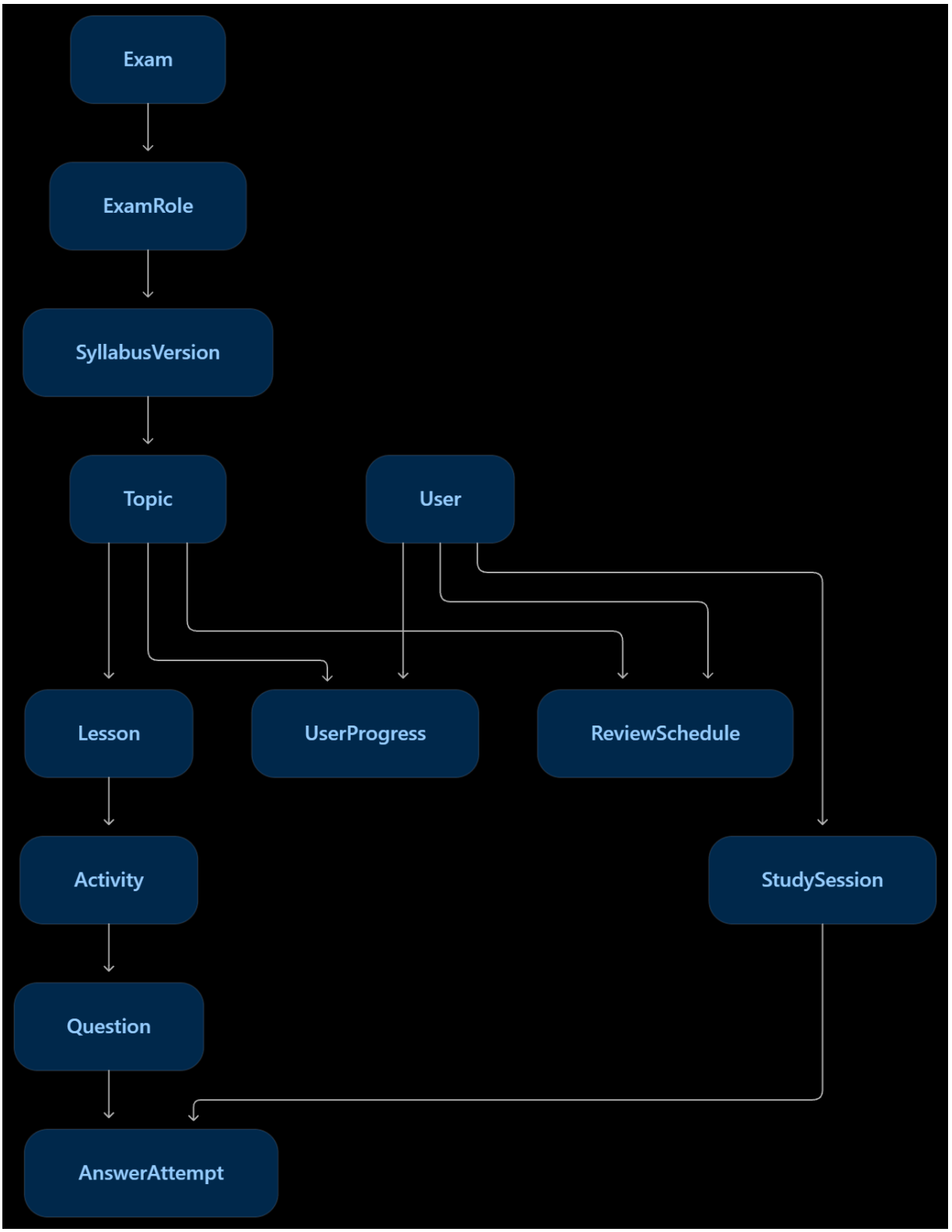
Recomendo monólito modular, em vez de microsserviços. Isso reduz a complexidade inicial, mas mantém domínios separados e permite demonstrar boas práticas arquiteturais.

## Organização do repositório

```
focaquest/
├── apps/
│   ├── web/                # Next.js
│   └── api/                # Spring Boot
│       └── src/
│           ├── main/
│           │   ├── java/.../focaquest/
│           │   │   ├── auth/
│           │   │   ├── catalog/
│           │   │   ├── learning/
│           │   │   ├── questions/
│           │   │   ├── progress/
│           │   │   ├── review/
│           │   │   └── shared/
│           │   └── resources/
│           │       └── db/migration/
│           └── test/
├── content/
│   └── tjpb-2026/
│       ├── syllabus/
│       ├── lessons/
│       └── questions/
├── docs/
│   ├── prd.md
│   ├── architecture.md
│   ├── data-model.md
│   ├── adr/
│   └── api/
├── infra/
│   └── docker-compose.yml
├── .github/
│   └── workflows/
├── README.md
└── LICENSE
```

## Modelo de dados inicial

Diagrama conceitual simplificado — as cardinalidades detalhadas e entidades associativas serão definidas na modelagem lógica.



### Primeiros endpoints da API

| Método | Endpoint | Finalidade |
|---|---|---|
| GET | `/api/v1/exams` | Listar concursos |
| GET | `/api/v1/exams/{id}/roles` | Consultar cargos |
| GET | `/api/v1/roles/{id}/syllabus` | Consultar trilhas e assuntos |
| GET | `/api/v1/lessons/{id}` | Carregar microlição |
| POST | `/api/v1/study-sessions` | Iniciar sessão |
| POST | `/api/v1/study-sessions/{id}/answers` | Registrar resposta |
| PATCH | `/api/v1/study-sessions/{id}` | Pausar ou retomar |
| GET | `/api/v1/me/progress` | Consultar progresso |
| GET | `/api/v1/me/reviews` | Consultar revisões |
| POST | `/api/v1/me/reviews/{id}/complete` | Concluir revisão |

Nas respostas de questões, o gabarito e a explicação não deverão ser enviados ao cliente antes da resolução, para preservar a integridade das tentativas.